WebSockets and real time communication

A PHP application for conversations that move in both directions.

Shared rooms, stored messages, visible connection state.

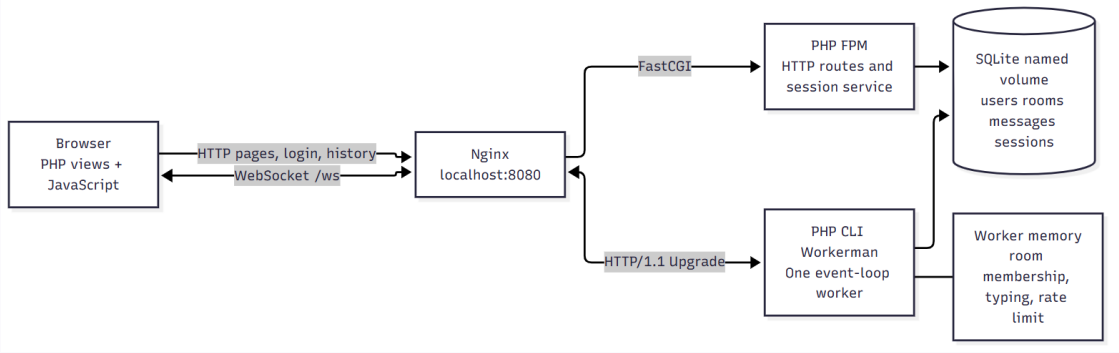
Team members _____

HTTP and persistent communication

HTTP remains useful for login, room discovery and history.

WebSocket lets either endpoint initiate a message over the established channel.

Chat, typing and presence use named JSON events.



Handshake identity and message boundaries

HTTP/1.1 Upgrade → validate Origin and session → 101

Workerman handles RFC 6455 frames, fragments and ping/pong.

The application validates JSON, identity and room membership.

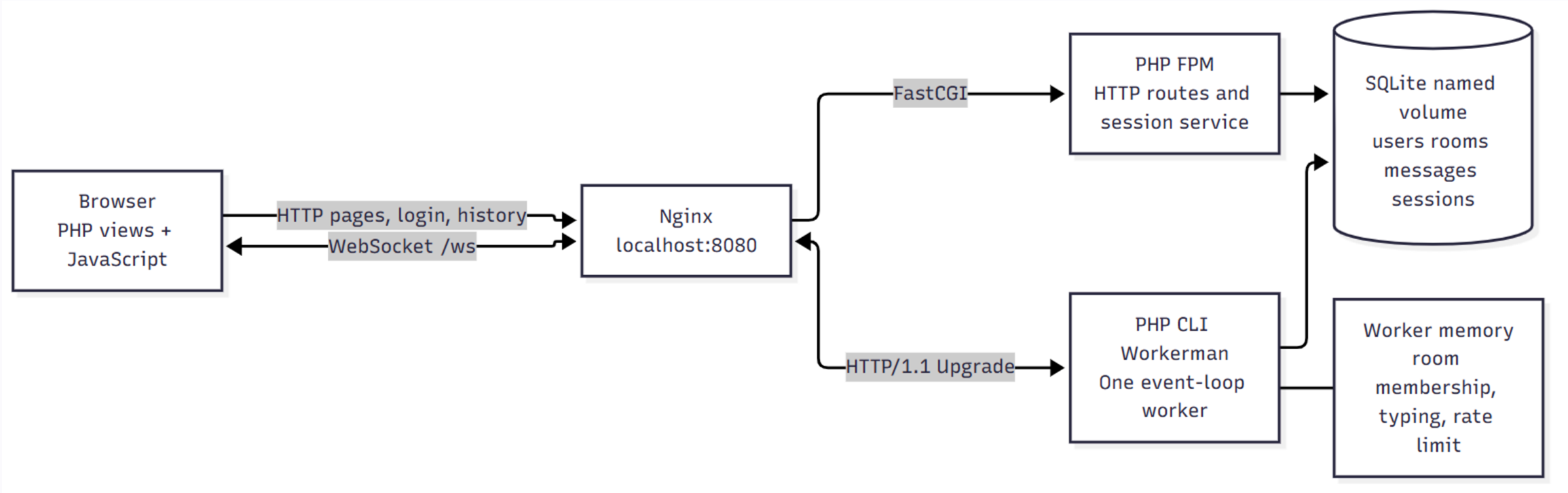
Masking is not encryption. Public deployment needs TLS and wss.

Transport alternatives

Transport	Delivery model	A useful fit
Short polling	Repeated HTTP checks	Infrequent changes
Long polling	Held request, then reissue	Streaming fallback
SSE	Server to browser stream	Feeds and progress
WebSocket	Persistent bidirectional messages	Interactive conversation

Our choice: WebSocket for live interaction; HTTP for finite operations.

One origin two PHP processes



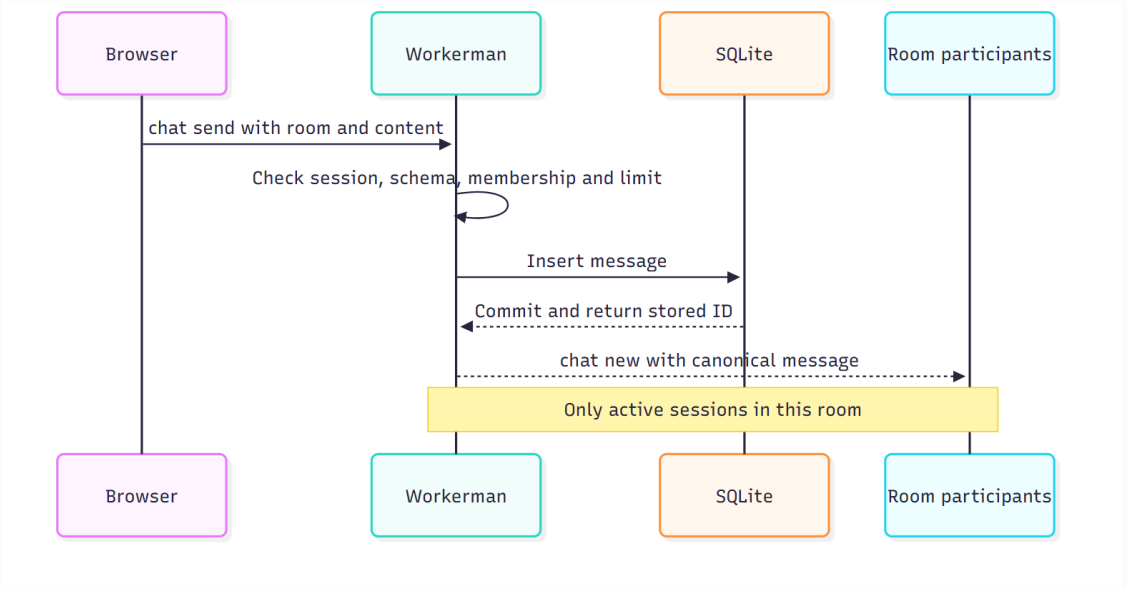
Nginx is the only published service. SQLite sessions connect the HTTP and WebSocket identity model.

Stored data and event flow

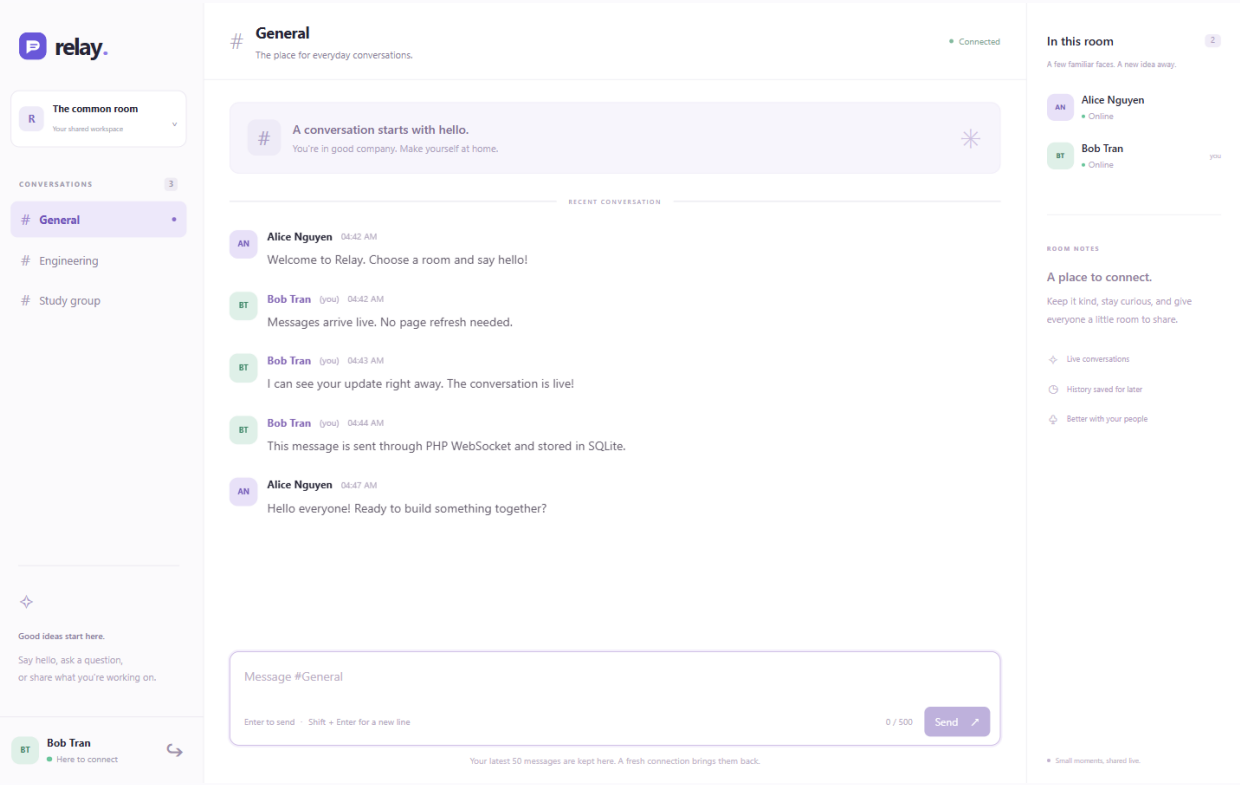
users / rooms / messages / sessions

Validate → check membership and account limit → insert → broadcast

A stored message ID is shared by HTTP history and live delivery.



Live room conversation



Two browser profiles. Two accounts. Matching message IDs without a page reload.

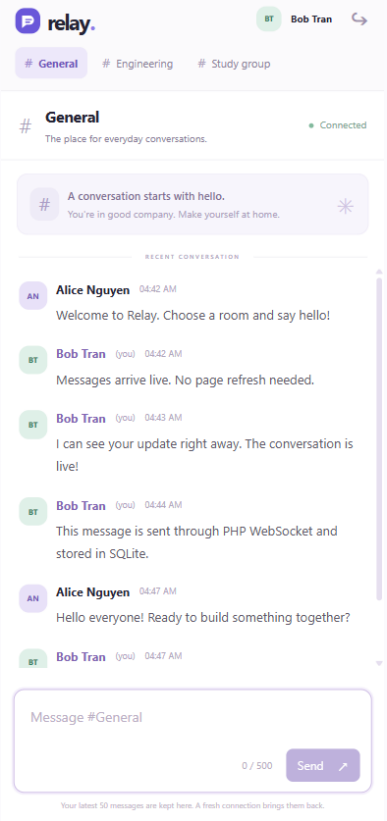
Presence typing and connection recovery

One account with two tabs still appears once.

Typing expires after three seconds.

Reconnect rejoins and merges the newest 50 messages by ID.

Uncertain sends are never automatically repeated.

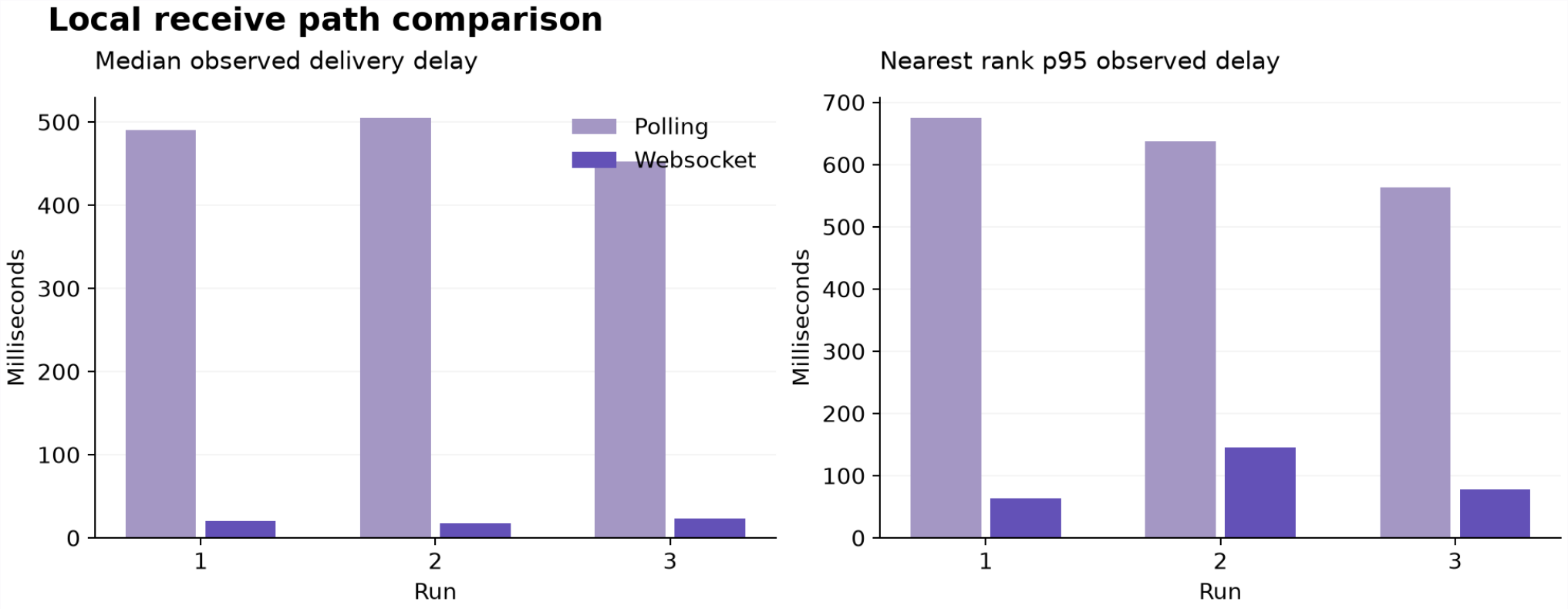


Persistence and defensive behavior

Check	Observed outcome
PHPUnit	28 tests / 78 assertions passed
HTTP and WS integration	7 of 7 groups passed
Restart	Stored marker remained
No pong client	Closed after 55.828 seconds
Room / payload / rate limit	Invalid operations rejected

Tests use an isolated stack. Browser checks supplement protocol checks.

Local polling and WebSocket results



Three 120-second runs per transport. 20 messages per run. All 120 received.

Polling median: 452.95–504.52 ms / WS median: 17.63–23.73 ms.

Results within clear limits

Functional fanout passed at 2, 10 and 20 simultaneous sockets.

One worker owns transient state. SQLite writes remain synchronous.

Latest-50 history is bounded recovery, not exactly-once delivery.

Local HTTP/WS is verified; public TLS and production capacity are not.

Application reliability beyond a socket

Identity, room authorization, storage and recovery are application responsibilities.

Next steps: durable replay cursor, idempotent send IDs, shared multi-worker state and verified WSS.

Source, setup commands, test evidence and raw measurements accompany the report.